Computer Graphics Notes

For BCA 4 Year Program

Prepared by:- Damanjeet Kaur

Unit 1

Overview of computer graphics and its applications, Historical development, Application areas e.g. entertainment, scientific visualization, user interfaces, Basics of raster and vector graphics, differences between raster and vector graphics, Representation and storage formats, Graphics pipeline and rendering process, stages of the graphics pipeline (modeling, transformation, rendering), concepts of rendering primitives, vertices, and fragments.

Computer graphics (CG) is the field of study that focuses on the creation, manipulation, and storage of geometric objects and images using computers

. It serves as a vital communication bridge, allowing data to be represented visually for easier human interpretation.

Historical Development

The evolution of CG began in the **1950s** with military projects like **Whirlwind** and **SAGE**, which used Cathode Ray Tubes (CRTs) and light pens for interaction.

- **1960s**: Ivan Sutherland's **Sketchpad (1963)** revolutionized interactive graphics. The term "computer graphics" was coined at Boeing in 1960.
- **1970s**: Foundational algorithms for **shading** (Phong, Gouraud) and the first **framebuffers** for raster graphics emerged.
- **1980s**: Personal computers like the **Macintosh** brought GUIs to consumers, and the first **GPUs** (like NEC's) were developed.
- **1990s-Present**: 3D graphics went mainstream with **Toy Story (1995)**, standards like **OpenGL/DirectX**, and the rise of real-time ray tracing in the late 2010s.

Application Areas

Computer graphics is integral across various sectors:

- **Entertainment**: Used for CGI in movies (special effects, animation) and interactive real-time graphics in video games.
- **Scientific Visualization**: Helps researchers visualize complex datasets, such as molecular structures, weather patterns, and medical MRI/CT scans.
- **User Interfaces (UI)**: Provides the graphical elements (icons, windows, menus) that make software intuitive and accessible.
- **Computer-Aided Design (CAD)**: Enables engineers and architects to create precise 3D models of buildings and machinery.

1. Raster Graphics (Bitmaps)

Raster images are like mosaics made of tiny squares called **pixels**.

- **Composition:** Each pixel is assigned a specific color value. When viewed together, these millions of pixels form a complete image.
- **Scalability:** They are **resolution-dependent**. If you enlarge a raster image, the pixels stretch, leading to a "pixelated" or blurry appearance.
- **Best For:** Photographs and complex digital paintings that require subtle color transitions and fine detail.
- **Common Formats:** JPEG, PNG, GIF, TIFF, BMP.
- **Software:** Adobe Photoshop, GIMP, Corel PHOTO-PAINT.

2. Vector Graphics

Vector images are created using mathematical paths defined by points, lines, and curves.

- **Composition:** Instead of storing pixel data, the file stores instructions (e.g., "draw a circle with a 5cm radius at these coordinates").
- **Scalability:** They are **resolution-independent**. You can scale them infinitely (from a business card to a billboard) and the edges will remain perfectly sharp because the math just recalculates at the new size.
- **Best For:** Logos, icons, typography, and technical drawings where clean lines and scalability are essential.
- **Common Formats:** SVG, AI, EPS, PDF.
- **Software:** Adobe Illustrator, CorelDRAW, Inkscape.

Comparison Summary

Feature	Raster Graphics	Vector Graphics
Basic Unit	Pixels (colored dots)	Mathematical paths
Scalability	Limited; loses quality when enlarged	Infinite; stays sharp at any size

File Size	Generally larger (depends on resolution)	Generally smaller (depends on complexity)
Common Use	Photos, detailed textures	Logos, icons, fonts, illustrations
Conversion	Hard to convert to vector (tracing)	Easy to convert to raster (rasterization)
Composition	Made of a grid of tiny colored dots called **pixels**.	Defined by **mathematical formulas**, lines, and curves.
Scalability	Becomes pixelated or "jagged" when enlarged (aliasing).	Infinitely scalable without any loss of quality or clarity.
Detail	Ideal for high-detail photos and subtle color gradations.	Best for logos, icons, and geometric illustrations.
File Size	Generally larger; size grows with resolution/pixel count.	Much smaller; only stores mathematical data for paths.
Formats	**JPEG, PNG, GIF, BMP, TIFF**.	**SVG, PDF, AI, EPS**.

Interpolation

Interpolation in computer graphics is the technique of calculating unknown intermediate values between two or more known data points to create smooth transitions, shapes, or images. It is used to generate pixel colors, animate between keyframes, resize images, and map textures, essentially filling in gaps to produce a continuous output.

Key Uses in Computer Graphics

- **Animation (Keyframing):** Filling in frames between keyframes to create smooth motion.
- **Rendering (Shading):** Determining colors across a polygon (e.g., Gouraud shading) based on vertex colors.
- **Image Resizing:** Calculating pixel values when scaling (upscaling/downscaling) or warping images.

- **Texture Mapping:** Mapping textures onto 3D surfaces and interpolating texels to avoid pixelation.
- **Digital Zoom:** Stretching an image by estimating values between pixels.

Common Types of Interpolation

- **Linear Interpolation (LERP):** The simplest form, connecting two points with a straight line to compute an intermediate value, often using a parameter between 0 and 1.
- **Bilinear Interpolation:** Extends linear interpolation to two dimensions, commonly used in texture mapping and resizing, considering the four nearest pixels.
- **Trilinear Interpolation:** Extends to three dimensions, often used for 3D textures.
- **Spline Interpolation (e.g., Cubic):** Uses curves to create smoother, more natural transitions than linear methods.
- **Nearest Neighbor:** Uses the value of the closest known data point; it's fast but produces blocky results.

Fragment

In computer graphics, a **fragment** is essentially a "potential pixel". It is the data generated during the **Rasterization** stage that determines if and how a specific point on the screen should be colored.

While a pixel is a physical dot on your screen, a fragment is a collection of state data—like depth, color, and texture coordinates—that must pass several "survival tests" before it earns the right to become that pixel.

What is inside a Fragment?

A fragment carries all the information needed for the final stages of rendering:

- **Window Space Position**: The exact(X,Y) coordinate on the screen.
- **Depth (Z-value)**: How far the fragment is from the camera, used to decide if it’s hidden behind another object.
- **Interpolated Attributes**: Colors, normals (for lighting), and texture coordinates that were calculated by blending the values from the 3D triangle's corners.
- **Alpha Value**: The transparency level (0.0 to 1.0).

The Life of a Fragment (Detailed Steps)

1. **Generation (Rasterization)**: The GPU takes a 3D triangle and determines which screen pixels it covers. For every covered pixel area, it creates a fragment.
2. **Fragment Shading**: The programmable Fragment Shader runs. It uses the fragment’s data to calculate its final color, applying textures and complex lighting.

3. **Early-Z Testing (Optimization)**: To save power, modern GPUs often check the **Depth** of a fragment *before* shading it. If it's clearly behind something else, it's discarded immediately.
4. **Culling & Stencil Tests**:
    1. **Scissor Test**: Discards fragments outside a specific rectangular "scissor" area on the screen.
    2. **Stencil Test**: Uses a "mask" to block rendering in certain shapes.
5. **Depth Testing**: The final check to see if the fragment is the closest one to the viewer. If it passes, its depth value is saved to the Depth Buffer.
6. **Blending**: If the object is transparent, the fragment's color is mixed with the existing background color in the Framebuffer.
7. **Final Output**: The fragment is finally written to memory as a colored **pixel** ready for display. Youngdo Lee +10

Visual Comparison: Fragment vs. Pixel

Feature	Fragment	Pixel
State	Incomplete / Potential	Final / Displayed
Location	Screen space + Depth (Z)	Screen space (X,Y)
Multiplicity	Can be many per pixel (Multisampling)	One per screen unit
Data	Includes normals, textures, depth	Final Color (RGB)

The Graphics Pipeline & Rendering Process

The graphics pipeline is the sequence of steps a GPU follows to transform 3D data into a 2D image on a screen.

1. **Modeling (Application Stage)**: The CPU defines the scene, including the geometry of objects, light sources, and camera position.
2. **Transformation (Geometry Stage)**:
1. **Vertices**: The basic points in 3D space.

2. **Vertex Processing**: Vertices are transformed from their local "object space" to "world space" and finally "screen space" using mathematical matrices.
3. **Rendering (Rasterization & Shading)**:

0. **Primitives**: Basic shapes (usually triangles) formed by connecting vertices.
1. **Rasterization**: The GPU determines which screen pixels are covered by each primitive.
2. **Fragments**: These are "potential pixels" generated during rasterization, containing data like color, depth, and texture.
3. **Fragment/Pixel Shading**: The final color of each fragment is calculated, often using textures and lighting models, before being written to the framebuffer for display

<u>Primitives</u>

In computer graphics, **graphics primitives** are the fundamental geometric building blocks used to construct complex scenes and images.

1. Types of Primitives

Primitives are low-level objects from which higher-level graphics are built. The most common types include:

- **Lines**: Straight paths connecting two points (vertices). They are used for wireframe models, charts, and outlines.
- **Polygons**: Closed figures made of three or more line segments (edges) joined at vertices. **Triangles** are the most common primitive used in modern GPUs because they are always planar and easy to rasterize.
- **Points**: Represent a single position in space defined by coordinates (e.g. X,Y in 2D). On a screen, a point is often represented by a single pixel.

2. Coordinate Systems

Coordinate systems provide a mathematical way to assign numbers to points in space.

Cartesian Coordinate System

This is the standard (3D) system based on perpendicular axes. Points are defined by their signed distances from these axes.

- **Pros**: Highly intuitive for measuring physical distances.
- **Cons**: Certain transformations, like translation (moving an object), cannot be represented as simple matrix multiplications; they require vector addition

Homogeneous Coordinate System

This system adds an extra dimension (typically called X,Y) to represent points. A 2D point becomes (X,Y) , and a 3D point becomes (X,Y,Z)

- **Unified Transformations**: Allows all transformations (translation, rotation, scaling) to be performed using only matrix multiplication
- **Points at Infinity**: Can represent infinitely distant points, which is crucial for perspective projections.
- **Matrix Concatenation**: Multiple transformations can be "baked" into a single matrix by multiplying them together, significantly reducing computational overhead during rendering.

3. Transformation Matrices & Applications

Transformation matrices are

Transformation	Purpose	Mathematical Application
Translation	Shifts an object to a new location.	Moves a model to its correct world position.
Scaling	Changes the size of an object.	Uniform or non-uniform resizing of models.
Rotation	Pivots an object around an axis.	Orienting a character to face a specific direction.
Shearing	Slants an object by shifting its vertices proportionally.	Creating "italic" text or simulated 2D depth.
Projection	Maps 3D coordinates onto a 2D screen.	Simulates perspective (objects getting smaller with distance).

Representation and storage formats in computer graphics

Computer graphics are represented and stored through various data structures and file formats, primarily categorized into **Raster**, **Vector**, and **Metafile** types. The choice of format affects image quality, file size, and the ability to scale or edit the content.

Adobe +1

1. Graphic Representation Methods

- **Raster (Bitmap) Representation:** Images are represented as a rectangular grid (matrix) of numeric values called **pixels**. Each pixel stores specific color information based on its intensity and bit depth.
- **Vector Representation:** Images are defined by **mathematical equations** and geometric primitives like points, lines, and curves. Instead of individual dots, the computer stores instructions to "draw" the shapes.
- **Metafile Representation:** These are hybrid formats that can encapsulate **both raster and vector data** in a single file.

2. Common Storage Formats

Raster Storage Formats

Raster formats are primarily used for photographs and detailed imagery.

- **JPEG (.jpg):** Uses **lossy compression**, making it ideal for web photos where small file size is more important than perfect quality.
- **PNG (.png):** A **lossless** format that supports transparency (alpha channels). It is widely used for web graphics and logos.
- **GIF (.gif):** Limited to **256 colors** but supports simple animations. It uses lossless LZW compression.
- **TIFF (.tif):** A flexible, high-quality format used in professional printing and archiving. It can be uncompressed or use lossless compression.
- **BMP (.bmp):** A standard Windows format that is typically **uncompressed**, resulting in very large files but no quality loss.
- **RAW:** Unprocessed data directly from a camera sensor, containing the maximum amount of image information for professional editing.

Vector Storage Formats

Vector formats are preferred for logos, typography, and technical drawings.

- **SVG (.svg):** An XML-based open standard for 2D vector graphics, commonly used in web design for its infinite scalability.
- **AI (.ai):** The proprietary native format for **Adobe Illustrator**.
- **EPS (.eps):** A standard format for high-resolution graphics and illustrations used in the printing industry.

Compound & 3D Formats

- **PDF (.pdf):** A universal container that can store text, raster images, and vector graphics while maintaining consistent layout across devices.
- **glTF / OBJ / STL:** Specialized formats for **3D graphics**, storing surface geometry, textures, and material properties.

3. Key Storage Characteristics

- **Compression:**
  - **Lossy:** Permanently removes redundant data to significantly reduce file size (e.g., JPEG).
  - **Lossless:** Reduces file size without any data loss, allowing the original image to be perfectly reconstructed (e.g., PNG, GIF).
- **Color Depth:** Refers to the bits used per pixel. Common depths include 8-bit (256 colors), 24-bit (True Color), and 32-bit (True Color plus transparency).

<u>Culling And Clipping</u>

In computer graphics, **culling** and **clipping** are both optimization techniques used to remove invisible or unnecessary geometry to improve rendering performance.

The fundamental difference is that **culling completely removes objects** that are not visible (such as those behind the camera), while **clipping cuts objects** that are partially visible at the edges of the screen.

Culling (Removal of Unseen Objects)

Culling is an early optimization step that discards entire objects or primitives (triangles) that do not contribute to the final image before they go through heavy, detailed processing.

- **Purpose:** To prevent the GPU from wasting time processing vertices and fragments that won't be seen.
- **When it happens:** Generally done early in the pipeline (application phase).

- **Types:**
  - **Frustum Culling:** Removing objects outside the camera's viewing field.
  - **Back-face Culling:** Removing polygons that face away from the camera.
  - **Occlusion Culling:** Removing objects hidden behind other objects.

Clipping (Cutting Partially Seen Objects)

Clipping acts as a secondary stage, refining the geometry that passed through culling. It keeps only the parts of primitives that fall within the viewable area, often creating new vertices in the process.

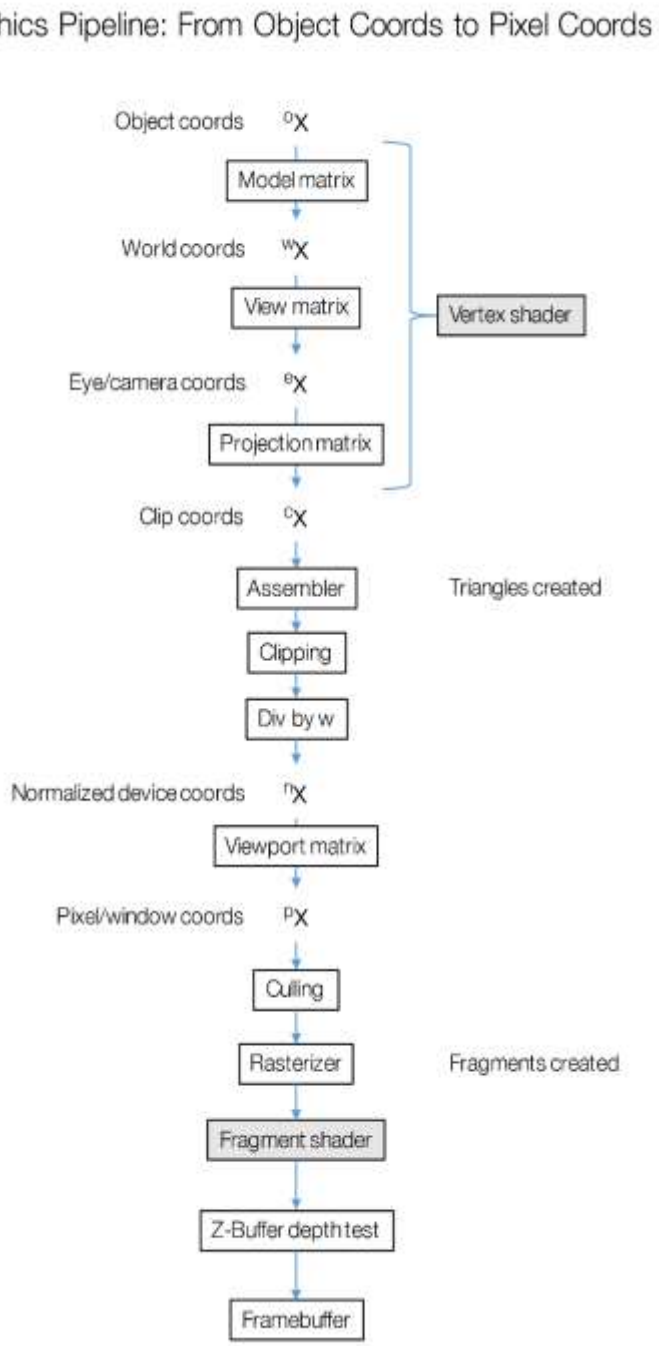
- **Purpose:** To clip primitives that span across the viewing volume boundaries, ensuring they fit within the screen.
- **When it happens:** Generally done just before rasterization.
- **Function:** If a triangle is half-inside and half-outside the screen, clipping cuts it and generates new vertices to form a smaller triangle that is completely visible.

Summary of Differences

Feature	Culling	Clipping
Action	Discards entire objects	Cuts primitives partially inside/outside
Visibility	Removes things totally hidden	Handles objects on the boundary
Output	Fewer primitives	Potentially more vertices/primitives
Pipeline Stage	Early (Application/Transform)	Late (Before Rasterization)
Complexity	Simple, fast, large speedup	Complex, slower, per-pixel handling

Analogy

- **Culling** is like taking a photograph and deciding not to take a picture of a house because it is entirely behind you.
- **Clipping** is like taking a picture and having the edge of the photo cut through a tree—you only see the part of the tree that fits inside the photo frame.



Graphics Rendering

Understanding the graphics rendering pipeline is essential for designing or developing 3D and immersive experiences. It helps you make informed decisions and collaborate effectively with engineers to create designs that are both beautiful and efficient. This guide offers a high-level,

beginner-friendly overview of the graphics rendering pipeline for the Meta Quest headset and other supported devices. Here's why understanding the graphics rendering pipeline matters:

- **Performance optimization**: The VR headset has limited processing power compared to high-end PCs or consoles. Each pipeline stage uses resources, so optimizing assets and effects is crucial. For example, minimizing complex geometry and using optimized textures help maintain smooth frame rates and reduce latency, improving user experience. See Balancing art and performance for immersive experiences for more insights.

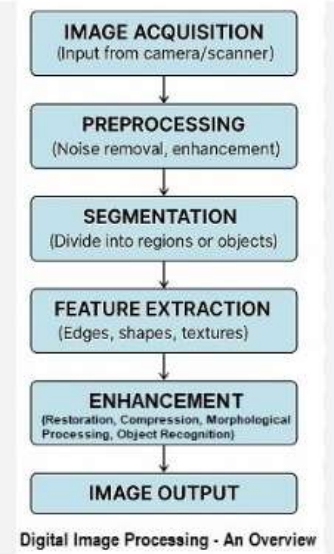
- **Visual quality**: Understanding how graphics render lets you make smart choices about lighting, shading, and texturing. This improves visual fidelity and immersion. For example, knowing shading techniques helps create realistic materials, while understanding rasterization helps avoid visual artifacts.

  Understanding the graphics rendering pipeline offers distinct advantages to designers, developers/engineers, and digital artists.

- **For designers**, knowledge of the pipeline enables strategic planning of assets and visual effects, ensuring that creative decisions are made with performance constraints in mind and have the greatest possible impact on the final user experience.

- **Developers and engineers** benefit by using the pipeline's structure as a reference point when implementing, optimizing, or debugging rendering features. This understanding makes it easier to identify bottlenecks, streamline workflows, and improve overall efficiency.

- **3D and 2D digital artists** gain valuable insight into how models and textures are processed at each stage of the pipeline. This allows them to enhance detail and style where it matters most, while also ensuring their work remains compatible and performs well across different devices. By appreciating the pipeline's role in rendering, each discipline can collaborate more effectively and contribute to creating visually stunning and high-performing immersive experiences.



Graphics rendering

Graphics rendering is the process of turning digital information into images that you see on your screen. Below are terms that will be useful to know.

Terminology

Term	Definition
Graphics Rendering Pipeline	The graphics rendering pipeline is a sequential process that transforms 3D model data into a visually compelling 2D image by progressively refining geometry, applying lighting and effects, and optimizing for performance at each stage.
Raster	A grid of pixels that forms an image, commonly used in digital graphics and displays.
2D	Two-dimensional graphics, representing images with only height and width (no depth).
3D	Three-dimensional graphics, representing objects with height, width, and depth for a more realistic appearance.

Term	Definition
Vertex	A point in space that defines the corners or intersections of geometric shapes in graphics.
X, Y, Z Axises	The X, Y, and Z axes are the three imaginary lines used to define positions in 3D space: X-Axis is the horizontal line, usually representing left and right. Y-Axis is the vertical line, usually representing up and down. Z-Axis is the depth line, usually representing forward and backward. Together, these axes form a 3D coordinate system, allowing you to specify any point in space using three values (X, Y, Z)
Tessellation	The process of dividing a surface or shape into smaller geometric pieces, often triangles, to improve rendering detail.
Rasterization	The process of converting shapes (like meshes) into pixels for display on a screen.
GPU (Graphics Processing Unit)	A specialized hardware component designed to rapidly process and render graphics and images.
CPU (Central Processing Unit)	The Central Processing Unit; the main processor in a computer responsible for general-purpose computing tasks.
Pixels	A pixel is the smallest unit of a digital image, representing a single point of color on a screen.
Shaders	Small, specialized programs that run on the GPU.

Term	Definition
Polygons	In 3D graphics, polygons (usually triangles or quadrilaterals) are used to build the surfaces of 3D models.
Wireframes	Visual representations of 3D models that show only the edges and vertices, resembling a “skeleton” of the object. They help visualize the structure and shape without rendering surfaces or textures.
Mesh	A mesh is a collection of vertices, edges, and faces (often polygons) that define the shape of a 3D object. Meshes are the fundamental building blocks for 3D modeling and animation.
Camera View Frustum	A 3D volume representing what the camera can see.

Tessellation

Tessellation is a powerful GPU-based technique that dynamically subdivides 3D surfaces into smaller primitives, typically triangles, to create smoother and more detailed geometry in real-time. By generating this detail directly on the graphics hardware, applications can use simpler, low-polygon "base" models while rendering high-resolution visuals, significantly saving memory and bandwidth.

Aliasing

Image aliasing is a visual phenomenon in digital imaging where a sampled representation of an image exhibits undesirable artifacts or distortions. These artifacts typically appear as jagged or “stair-stepped” edges, which can significantly reduce the overall quality and detail in the image.

The primary cause of image aliasing is the limitations of digital sensors or screens, which are composed of discrete pixels. When an image is captured or displayed with a limited number of pixels, high-frequency content (such as fine lines, textures, or intricate patterns) can become difficult to resolve, resulting in aliasing.

Aliasing is not limited to a single visual artifact; it also includes effects like Moiré patterns (strange wave-like or swirl patterns) and color noise (random color changes). Furthermore, aliasing can occur not only in space (spatial aliasing) but also in time (temporal aliasing), such as in videos or animations, where motion appears jerky or unnatural. These issues arise due to the misinterpretation of frequencies that are higher than half the sampling rate, causing them to be perceived as lower frequencies. This phenomenon, a result of under-sampling, can make signals or details indistinguishable, leading to measurement errors in digital imaging.

In computer graphics, a specific type of aliasing occurs when diagonal lines or curves appear stair-stepped due to insufficient pixel density to represent these lines realistically. This is commonly referred to as “stair-stepping” or “jaggies” in computer graphics.

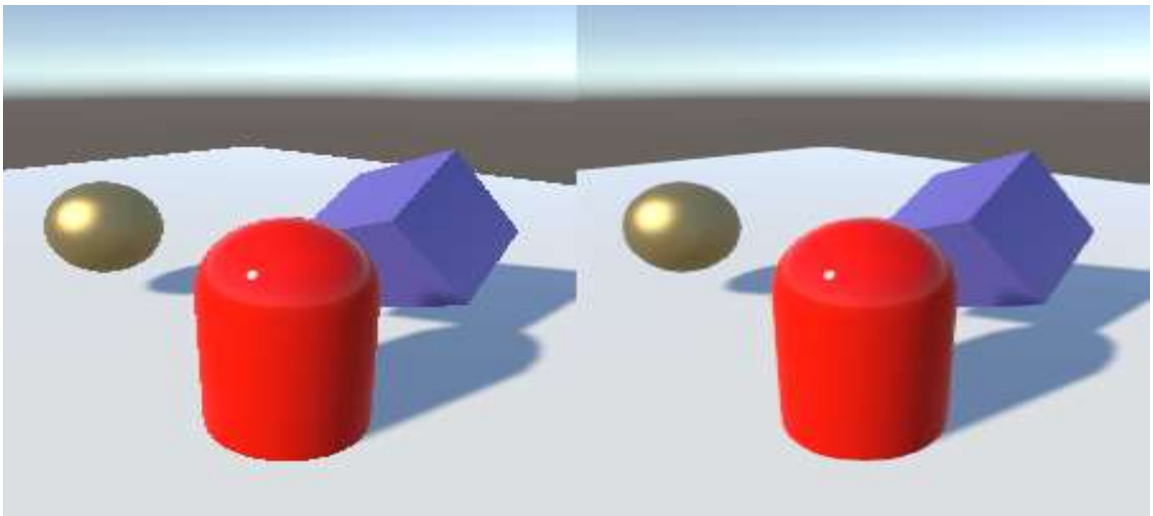
In berief, Aliasing is a visual distortion that occurs when a high-resolution or continuous signal is represented at a lower resolution, typically resulting in jagged, "staircase" edges known as **jaggies**.

Anti-aliasing (AA) is the collection of techniques used to mitigate these distortions by smoothing out the jagged edges to create a more natural and realistic image.

Key Differences

Feature	**Aliasing**	**Anti-Aliasing**
Appearance	Sharp, jagged "staircase" edges on diagonal lines and curves.	Smooth, blended transitions that simulate a continuous line.
Cause	**Undersampling**: The screen resolution is too low to capture the fine detail of the 3D model.	**Blending/Filtering**: Techniques that add "buffer" pixels of intermediate colors to soften edges.

Performance Impact	None; it is the "default" state of a low-resolution render.	Taxes the GPU, as it requires additional calculations to blend or oversample pixels.



Common Anti-Aliasing Techniques

- **Supersampling (SSAA)**: Renders the entire image at a much higher resolution and then shrinks it down to fit your screen. It provides the best quality but is very demanding.
- **Multisampling (MSAA)**: A more efficient version of SSAA that focuses only on the edges of objects rather than the whole screen.
- **Fast Approximate (FXAA)**: A fast post-processing filter that blurs the jagged edges. It's very light on performance but can make the whole image look slightly blurry.
- **Temporal (TAA)**: Compares data from previous frames to the current one to smooth out edges. It is very effective for objects in motion but can cause "ghosting" artifacts.

3D Texture Mapping Techniques

Texture mapping in animation involves various techniques to enhance the visual quality of 3D models. Different types of texture mapping serve specific purposes in creating more realistic and visually appealing animations:

Diffuse Mapping: This is the most basic type of texture mapping. It involves applying color and texture to the surface of 3D models, giving them their base appearance. In animation, diffuse maps are used to add basic colors, patterns, and textures to surfaces, making objects look more realistic by mimicking their real-world appearance. It is the most common texture map used in game design, 3D visualization, and animation to create a matte or basic surface appearance.

Normal mapping: Normal maps are used to simulate complex surface details without adding extra geometric complexity to the model. These maps encode surface-normal information, giving the illusion of surface detail and depth by affecting how light interacts with the surface. This is widely used in animation to create intricate textures, such as adding bumps, grooves, or fine surface details to objects without altering their geometry.

Specular mapping: Specular maps define the shininess or reflectivity of a surface. In animation, this type of mapping is used to determine areas that appear more reflective or shiny on an object. By adjusting the specular map, animators can control where highlights and reflections occur, adding realism and depth to the materials being represented.

Bump mapping: Like normal mapping, bump maps simulate surface details. However, instead of altering the direction of the surface, bump maps modify the surface's apparent depth, creating the illusion of bumps, dents, or roughness on a surface. This technique is widely used in animation to add texture and depth to surfaces without the need for additional geometry.

Mesh Generation

Mesh generation in computer graphics creates 3D models by discretizing complex shapes into smaller, simple elements—typically triangles or quadrilaterals—defined by vertices, edges, and

faces. It acts as the foundation for rendering, simulation, and CAD, turning continuous geometries into manageable digital structures (structured grids or unstructured, flexible meshes).

Key Concepts in Mesh Generation

- **Definition:** A mesh is a 3D model formed by vertices (points), edges (lines connecting vertices), and faces (filled polygons).
- **Purpose:** Meshes are essential for rendering objects on a screen, calculating physics, and CAD, where complex shapes are broken into manageable parts.
- **Types of Meshes:**
  - **Structured Mesh:** Uses a regular lattice (e.g., a rectangular array) with implicit connectivity.
  - **Unstructured Mesh:** Allows irregular connectivity, which is better for complex geometries and often composed of triangles/tetrahedra.
- **Elements:** Triangles are common in graphics, while tetrahedra or hexahedra are used for volumetric mesh generation.

Mesh Generation Techniques

1. **Procedural Generation:** Programmatically creating geometry during runtime, such as terrain or procedural buildings.
2. **Surface of Revolution:** Creating a mesh by rotating a profile line around an axis, like creating a cylinder or bottle.
3. **Marching Cubes/Tetrahedra:** Algorithms used to convert 3D data fields (like CT scans or neural representations) into a surface mesh.
4. **Delaunay Triangulation/Voronoi Diagrams:** Used to reconstruct meshes from scattered point cloud data, frequently applied in scanning technology.

Key Considerations

- **Quality:** Good meshes have well-shaped elements (not too thin or stretched) for accurate simulations, with equilateral triangles or right-angled quadrilaterals being ideal.
- **Refinement:** Fine-tuning the grid, especially at boundaries, is necessary for high-fidelity results.
- **Data Structures:** Efficient representation is crucial, often requiring optimized storage for large datasets in game engines or CAD software

How are graphics rendered?

Graphics are rendered by specialized hardware called a GPU (Graphics Processing Unit), which processes visual data and transforms it into images displayed on your screen. The GPU works alongside the computer's CPU, handling complex calculations to create smooth and detailed visuals in real time.

Graphics rendering pipeline

The graphics rendering pipeline is an abstracted sequence of steps that a computer graphics system uses to create a 2D image from 3D model data. It involves several stages, each stage transforms and processes data to ultimately render the final image on the screen.

Vertex processing

Vertex processing is when the raw 3D data of a model is prepared for display on a 2D screen. This process begins with each vertex, defined by its 3D coordinates (X, Y, Z), undergoing a series of mathematical transformations. These transformations include:

- **Translation**: Moving the object to a specific position in the scene.
- **Rotation**: Spinning the object around an axis.
- **Scaling**: Changing the size of the object.

After these transformations, the system projects the 3D coordinates onto the 2D plane of your display, determining exactly where each point will appear on the screen. During vertex processing, basic lighting calculations are also performed. This means the system estimates how light interacts with each vertex, which helps create the desired shading and highlights for the final image.

How does vertex processing happen?

The process is handled by specialized programs called vertex shaders, which run on the GPU and are optimized for speed and efficiency. Efficient management of vertex data is especially important for devices like the Quest headset, where hardware resources are limited. By organizing and processing vertex information smartly, developers can ensure smooth performance and high-quality visuals, even in complex scenes.

Tessellation

Tessellation builds directly on the work done during vertex processing. The goal is to add more geometric detail by subdividing large polygons into many smaller triangles, which allows for smoother curves and richer textures in the final image. After the initial stage where the CPU and GPU transform 3D coordinates and apply rotations, scaling, and lighting effects to each vertex, tessellation takes these processed vertices and further refines the model's surface.

How does tessellation happen?

During tessellation, the CPU acts as the coordinator, determining which parts of the scene need extra detail, often based on how close objects are to the viewer or the overall performance requirements. The CPU then sends these instructions to the GPU, which uses specialized tessellation shaders to break down the surfaces efficiently and in real time. This dynamic adjustment means that only the necessary areas receive extra detail, helping to balance visual quality and system performance.

The collaboration between vertex processing and tessellation is essential for modern graphics rendering. Vertex processing sets up the basic structure and transformations, while tessellation adds the fine detail that makes objects look realistic and visually appealing. Tessellation allows for smoother curves, richer textures, and more lifelike models, especially when objects are viewed up close. Designers and artists benefit from this technology because it automates the process of adding detail, freeing them to focus on creativity rather than technical limitations.

Geometry processing

Geometry processing is where the shapes and structures of 3D models are created, manipulated, and prepared for further rendering. After vertex processing and tessellation have established the basic positions and added detail to the model's surface, geometry processing takes over to handle more complex operations on these shapes.

How does geometry processing happen?

During this stage, the GPU uses geometry shaders to generate new shapes, modify existing ones, or even remove unnecessary geometry to optimize performance. For example, geometry processing can create additional features like shadows, outlines, or special effects by manipulating the model's structure in real time. The CPU provides instructions and scene data, but the GPU performs the heavy lifting, efficiently handling large amounts of geometric information.

Geometry processing is essential for adding creative and technical enhancements to 3D models. It allows designers and artists to achieve effects such as dynamic environments, animated objects, and visually rich scenes without manually adjusting every detail. By working in tandem with vertex processing and tessellation, geometry processing ensures that the final shapes are ready for the next steps in rendering, such as shading and rasterization, resulting in high-quality visuals and smooth performance.

Mesh generation

Mesh generation is where the wireframe structure of 3D objects is built. This process defines how surfaces are constructed from interconnected vertices and edges, forming a network of polygons, most commonly triangles, that make up the shape of each object.

How does mesh generation processing happen?

After geometry processing has created and manipulated the shapes, mesh generation organizes these shapes into a coherent structure that the GPU can efficiently render. The mesh acts as the skeleton of a 3D model, providing the framework upon which textures, colors, and lighting effects are applied in later stages.

The complexity of a mesh, meaning the number of vertices, edges, and polygons, has a direct impact on rendering speed and memory usage. Simple meshes with fewer polygons are faster to render and require less memory, while highly detailed meshes can create more realistic visuals but may slow down performance if not managed carefully.

For designers and artists, understanding mesh generation is important because it influences both the visual quality and the efficiency of their work.

Well-constructed meshes allow for smooth surfaces and detailed models, while also ensuring that scenes run smoothly on different hardware. By balancing mesh complexity with performance needs, creators can achieve high-quality results without overwhelming the system, making mesh generation a crucial step in the digital art and design workflow.

Mesh shading

Mesh shading is where shading calculations are applied to the mesh, determining how light interacts with the surfaces of 3D objects. After the mesh has been generated, defining the structure of the object with vertices, edges, and polygons, mesh shading uses mathematical models to simulate the effects of light, shadow, and color on each part of the surface.

How does Mesh shading processing happen?

During this process, the GPU evaluates how different light sources in the scene affect the appearance of the mesh. It considers factors such as the angle of the light, the material properties of the surface, and the position of the viewer. These calculations result in realistic highlights, shadows, and gradients, which give objects depth and a sense of three-dimensionality.

Mesh shading is essential for achieving the desired realism and artistic style in digital art and design. The way light interacts with surfaces can make objects look smooth, shiny, rough, or matte, and can dramatically influence the mood and visual impact of a scene. For designers and artists, mesh shading provides the tools to control the look and feel of their creations, ensuring that objects not only have the right shape but also the right appearance under different lighting conditions.

Shading

Shading is the process that determines the final color and appearance of each pixel on the screen by simulating how light interacts with surfaces, textures, and materials. After the mesh and its structure have been defined, shading uses information about lighting conditions, surface textures, and material properties (such as glossiness, roughness, or transparency) to calculate how each part of an object should look.

How does shading happen?

This is accomplished using shaders. Shaders perform complex calculations to create a wide range of visual effects, from realistic reflections and shadows to artistic styles like cartoon outlines or glowing surfaces. For example, a shader can make a surface appear shiny and reflective, or soft and matte, depending on the artist's intent and the material settings.Efficient shading is essential for both visual quality and performance. Well-written shaders allow designers and artists to achieve the desired look for their objects and scenes without slowing down the rendering process. By carefully balancing detail and efficiency, shading brings digital art to life, making objects appear vibrant, dynamic, and visually compelling.

Clipping and backface culling

Clipping and backface culling are essential techniques in 3D rendering that help optimize performance and resource usage. **Clipping** refers to the process of removing portions of objects that fall outside the camera's view frustum, ensuring that only visible elements are processed and

rendered. This prevents unnecessary calculations for objects that the user cannot see, which is especially important in complex scenes.

Backface culling further improves efficiency by discarding the faces of objects that are oriented away from the camera. Since these faces are not visible to the user, rendering them would be wasteful. Together, these techniques reduce the number of polygons and fragments that need to be processed, leading to faster rendering and smoother experiences. For designers, understanding and leveraging clipping and backface culling can help create visually rich environments without compromising performance, particularly in VR where every millisecond counts.

How does clipping and backface culling happen?

Clipping and backface culling are performed automatically by the GPU during the rendering pipeline. Clipping uses the camera's view frustum, a 3D volume representing what the camera can see, to check each polygon and discard any parts that fall outside this area.

Backface culling works by calculating the orientation of each polygon relative to the camera; if a polygon's *front* is facing away, it is skipped and not rendered. These operations are handled by built-in GPU algorithms and settings, often controlled by toggling options in your engine or graphics API (like OpenGL or DirectX). Designers and developers can enable or adjust these features to optimize scenes, ensuring only visible surfaces are processed, which saves resources and boosts performance.

Rasterization

Rasterization is a key stage in the graphics rendering pipeline where the 2D representation of a scene is converted into a raster image made up of individual pixels. After all the previous steps, such as vertex processing, tessellation, geometry processing, mesh generation, and shading, have defined the shapes, details, and colors of objects, rasterization takes this information and determines exactly how each object will be drawn on the screen.

How does rasterization happen?

During rasterization, the GPU translates the mathematical descriptions of shapes (like triangles and polygons) into a grid of colored pixels that make up the final image you see. This process decides which pixels belong to which objects and how they should be colored based on the shading and lighting calculations performed earlier. One important aspect of rasterization is handling aliasing, which refers to the jagged edges that can appear when diagonal or curved lines are drawn on a

pixel grid. Techniques like anti-aliasing are used during this stage to smooth out these edges and improve the overall visual quality.

For designers and artists, understanding rasterization is valuable because it directly affects how your designs appear on screen. By knowing how shapes are converted to pixels and how issues like aliasing are managed, you can make more informed decisions to achieve the desired look and clarity in your digital artwork.

Render-to-texture

Render-to-Texture is a technique where a scene or object is rendered onto a texture rather than directly to the screen. This texture can then be used for various advanced visual effects, such as reflections, shadows, dynamic lighting, or post-processing filters. For example, a designer might use render to texture to create a mirror in a virtual environment or to apply a blur effect to a specific area. While this approach enables creative and visually impressive effects, it also increases the rendering workload and can impact performance if overused. Designers should balance the use of render to texture with the overall performance budget, ensuring that effects enhance the experience without causing lag or frame drops.

How does Render-to-texture happen?

Render to texture is achieved by directing the GPU to draw a scene or object onto a texture instead of the main screen. This is done by setting up a framebuffer object (FBO) or similar resource in your graphics engine, which acts as a temporary canvas.

Once rendered, the resulting texture can be used as a material on other objects (like mirrors, screens, or portals), or for post-processing effects such as blurring, distortion, or color correction. Most modern engines (Unity or Unreal) provide built-in tools and scripts for render-to-texture workflows, allowing designers to create advanced effects without deep graphics programming. However, each render-to-texture operation adds to the GPU's workload, so it's important to use this technique strategically to avoid performance drops.

Fragment processing and alpha blending

Fragment processing is the stage in the graphics pipeline where the final color and attributes of each pixel are determined. This includes applying effects such as transparency, blending, lighting, and shading. Optimizing fragment processing is crucial for achieving smooth and realistic rendering, especially on devices like the Quest headset where hardware resources are limited. Designers should be mindful of the complexity of their materials and shaders, as heavy fragment

processing can slow down rendering and reduce frame rates. By simplifying shaders, minimizing overdraw, and carefully managing transparency and blending, designers can ensure that their scenes look great while maintaining high performance in VR.

Alpha blending is a rendering technique used to handle transparency in digital graphics. When objects overlap, alpha blending calculates the final color by combining the colors of the overlapping objects based on their transparency values (alpha channel). This is seen in **passthrough** and **transparency** effects, where the virtual objects are blended with the physical environment.

This allows for effects such as glass, smoke, or semi-transparent UI elements, which are visually appealing and can add depth and realism to a scene. However, designers should be aware that alpha blending is computationally expensive, especially in VR environments like Quest, because it requires additional calculations for each overlapping pixel. Excessive use of transparency can lead to performance bottlenecks, so it's important to use alpha blending judiciously and optimize transparent assets to maintain a smooth user experience.

How does fragment processing and alpha blending happen?

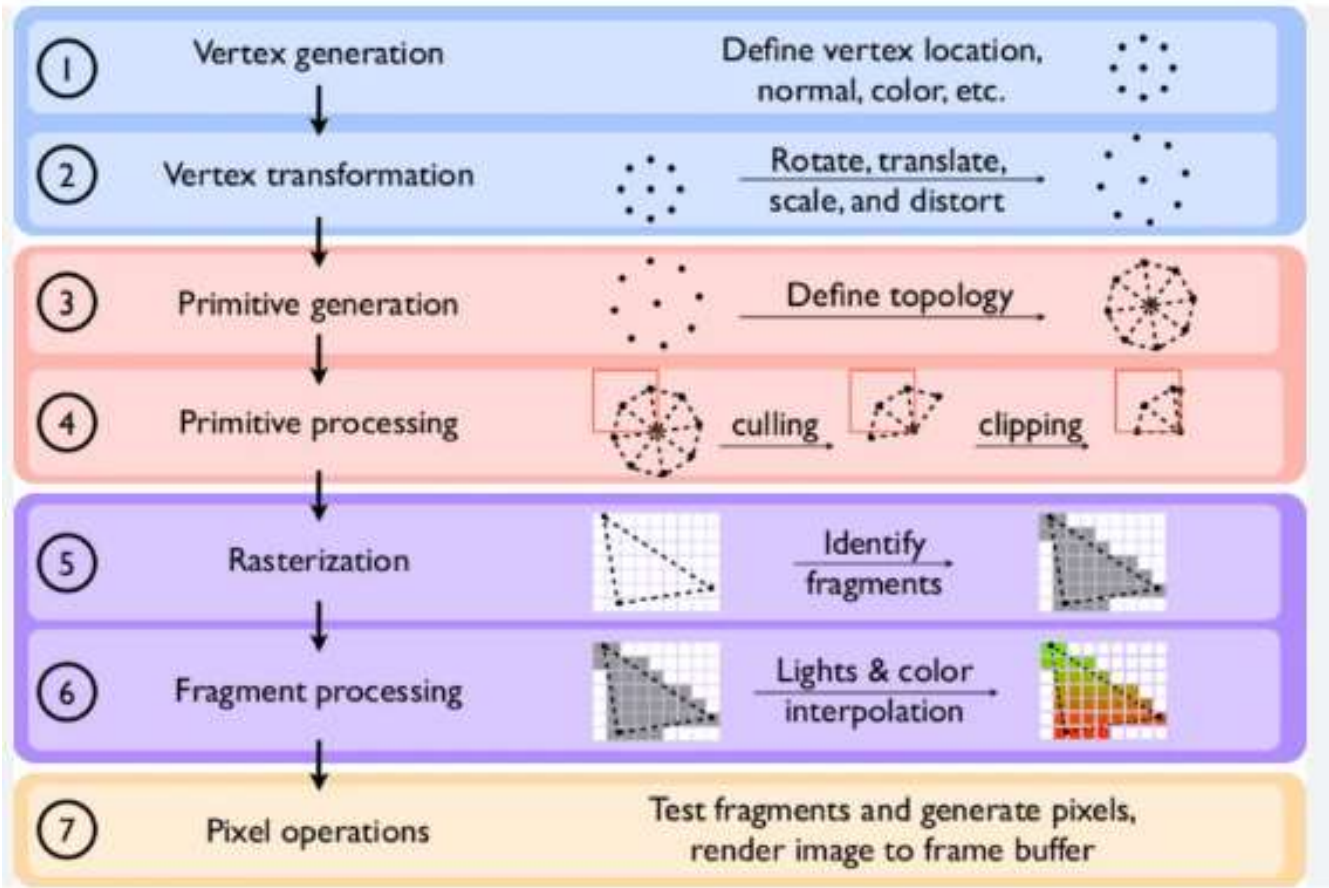
Fragment processing and alpha blending occur in the final stages of the graphics pipeline, handled by fragment shaders running on the GPU.

For each pixel (fragment) that will appear on the screen, the fragment shader calculates its color, lighting, and other properties based on material settings, light sources, and scene data. If transparency is involved, alpha blending combines the color of the fragment with the colors of any underlying pixels, using the alpha value to determine how much of each color should be visible. This blending is managed by the GPU's hardware and can be customized through shader code or engine settings. Designers can control transparency and blending modes in their materials, but should be aware that overlapping transparent objects require more calculations and can impact performance, especially in VR.

Graphics rendering pipeline key considerations

Stage	Main Function	Key Considerations
Vertex Processing	Transforms 3D model data and applies basic lighting to prepare for display.	Organize and optimize vertex data for smooth performance; use efficient transformations and lighting for quality visuals.
Tessellation	Subdivides polygons to add geometric detail for smoother, richer surfaces.	Balance added detail with system performance; use tessellation for close-up realism without overwhelming hardware.
Geometry Processing	Manipulates and generates geometry for effects and optimization.	Leverage geometry shaders for dynamic features; remove unnecessary geometry to maintain efficiency in complex scenes.
Mesh Generation	Builds the wireframe structure from vertices and edges.	Design meshes with appropriate complexity; ensure wireframes support desired detail while keeping rendering fast.
Mesh Shading	Calculates how light interacts with mesh surfaces for depth and realism.	Choose shading models that match artistic goals; optimize for realistic highlights, shadows, and gradients.
Shading	Determines final pixel color and appearance using lighting and material data.	Use shaders to achieve desired visual effects; balance realism and style with efficient shader code for smooth rendering.

Stage	Main Function	Key Considerations
Clipping and Backface Culling	Removes geometry outside the camera view or facing away to optimize rendering.	Apply these techniques to reduce hidden calculations; ensure only visible geometry is processed for better performance.
Rasterization	Converts 3D shapes into a 2D pixel grid for display.	Understand how pixel mapping affects image clarity; use anti-aliasing to smooth edges and enhance screen quality.
Render to Texture	Renders scenes or objects to textures for advanced effects.	Use for reflections, shadows, and post-processing; monitor performance impact and avoid excessive use in resource-limited VR.
Fragment Processing and Alpha Blending	Finalizes pixel color, transparency, and effects before display.	Optimize shaders and blending for realistic transparency; minimize overdraw and complex effects to maintain high frame rates.



Graphics Rendering in General

The graphics rendering pipeline is a multi-stage "assembly line" that transforms 3D data into a 2D image. The following image representations and descriptions explain each step in sequence:

1. Vertex Processing (Transformations)

The pipeline begins with Vertex Processing, where individual points (vertices) are moved, rotated, and scaled in 3D space.

- **Model to World Space**: Objects are placed at specific coordinates in the virtual scene.
- **View Projection**: 3D coordinates are projected onto a 2D plane based on the virtual camera's "view".

2. Tessellation & Geometry Processing

These stages add fine detail to the models before they are turned into pixels.

- **Tessellation**: Subdivides large triangles into smaller ones to create smoother curves, especially for close-up objects.

- **Geometry Shader**: Can generate new shapes or modify existing ones to create effects like shadows or outlines in real-time.

3. Rasterization

Rasterization is the critical bridge between 3D math and 2D pixels.

- **Triangle Setup**: The system identifies which pixels on your screen are covered by the 3D triangles.
- **Fragment Generation**: It creates a "fragment" (a potential pixel) for every covered spot, interpolating data like color and texture across the shape.

4. Fragment Shading (Pixel Coloring)

The Fragment Shader calculates the final color of each fragment.

- **Lighting & Textures**: It applies "wrapped" images (textures) and calculates how light, shadows, and reflections interact with the surface at a per-pixel level.

5. Output Merging (Tests & Blending)

The final step decides which colors actually appear on the screen.

- **Depth (Z) Testing**: Closest objects are kept, while hidden fragments (behind other objects) are discarded.
- **Alpha Blending**: Transparent objects, like glass or smoke, are mixed with the background colors to create a layered effect.
- **Framebuffer**: The final valid pixels are written to memory and displayed as a complete frame.

Unit -1 Questions

Practical Question

Implement a program to define and use basic graphics primitives such as points, lines, and polygons.

Theory Questions

1 What are the application areas of computer Graphics?
2 What are the difference in Raster and Vector Graphics approaches?
3 What are the different Representation and storage formats in Computer Graphics?
4 Define Graphics pipeline and rendering process with Example?
5 What are the stages of the graphics pipeline?
6 Explain the concepts of rendering primitives, vertices, and fragments?
7 Different between Culling vs Clipping, Aliasing vs Anti-aliasing?